



AI5707: PATTERN RECOGNITION AND MACHINE LEARNING

**Lab 2: Image Reconstruction
using Eigenvalue Decomposition
(EVD) and Singular Value
Decomposition (SVD)**

**KARTHIKEYAN P L 26120031
ADARSH M S 26120021**


```

        groups.append([i, pair])
        used.update([i, pair])
    else:
        groups.append([i])
        used.add(i)

# Arrange the pairs in descending order

groups.sort(
    key=lambda g: max(np.abs(eigval[g])),
    reverse=True
)

order = [i for group in groups for i in group]

eigval = eigval[order]
Q = Q[:, order]

# These are the k values chosen for the experiment

chosen_k = [10, 50, 100]

# Make sure a k does not split a conjugate pair

valid_k = []
count = 0

while count < n:

    if abs(eigval[count].imag) < 1e-6:
        count += 1
    else:
        count += 2

    valid_k.append(count)

ks = [
    min(valid_k, key=lambda x: abs(x - k))
    for k in chosen_k
]

print("EVD k values:", ks)

# Inverse of Q
Q_inv = np.linalg.inv(Q)

# Reconstruct the image
for k in ks:

    Lambda = np.zeros((n, n), dtype=complex)

```

```

Lambda[:k, :k] = np.diag(eigval[:k])

Ak = np.real(Q @ Lambda @ Q_inv)

error_img = np.abs(A - Ak)
error = np.linalg.norm(A - Ak, "fro")

print(f"k = {k}, Frobenius error = {error:.4f}")

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(A, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(np.clip(Ak, 0, 255), cmap="gray")
plt.title(f"EVD, k = {k}")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(error_img, cmap="gray")
plt.title("Error")
plt.axis("off")

plt.tight_layout()
plt.show()

# Error for all valid k values, as required in the assignment

errors = []

for k in valid_k:

    Lambda = np.zeros((n, n), dtype=complex)
    Lambda[:k, :k] = np.diag(eigval[:k])

    Ak = np.real(Q @ Lambda @ Q_inv)

    errors.append(
        np.linalg.norm(A - Ak, "fro")
    )

plt.figure(figsize=(8, 5))
plt.plot(valid_k, errors, marker="o")
plt.xlabel("k")
plt.ylabel("Frobenius Error")
plt.title("EVD Reconstruction Error")
plt.grid()

```

```
plt.show()
```

Question 2 — SVD of Square Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image and convert it to grayscale
img = cv2.imread("cat_16.png")
A = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(float)

# Do the SVD
U, S, Vt = np.linalg.svd(A, full_matrices=False)

# These are the k values chosen for the experiment
ks = [10, 50, 100]

print("SVD k values:", ks)

# Reconstruct the image for each chosen k
for k in ks:

    Ak = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]

    error_img = np.abs(A - Ak)
    error = np.linalg.norm(A - Ak, "fro")

    print(f"k = {k}, Frobenius error = {error:.4f}")

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.imshow(A, cmap="gray")
    plt.title("Original")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(np.clip(Ak, 0, 255), cmap="gray")
    plt.title(f"SVD, k = {k}")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(error_img, cmap="gray")
    plt.title("Error")
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

```
# Check the error for every possible k
errors = []
```

```
for k in range(1, len(S) + 1):
```

```
    Ak = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
```

```
    errors.append(
        np.linalg.norm(A - Ak, "fro")
    )
```

```
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(S) + 1), errors)
plt.xlabel("k")
plt.ylabel("Frobenius Error")
plt.title("SVD Reconstruction Error")
plt.grid()
plt.show()
```

Question 3 — SVD of Rectangular Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the rectangular image and make it grayscale
img = cv2.imread("cat_16.png")
A = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(float)

# SVD works directly on a rectangular matrix
U, S, Vt = np.linalg.svd(A, full_matrices=False)

# These are the k values chosen for the experiment
ks = [10, 50, 100]

print("SVD k values:", ks)

# Reconstruct the image for each chosen k
for k in ks:

    Ak = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]

    error_img = np.abs(A - Ak)
    error = np.linalg.norm(A - Ak, "fro")

    print(f"k = {k}, Frobenius error = {error:.4f}")

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.imshow(A, cmap="gray")
    plt.title("Original")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(np.clip(Ak, 0, 255), cmap="gray")
    plt.title(f"SVD, k = {k}")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(error_img, cmap="gray")
    plt.title("Error")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

# Error for every possible k
```

```
errors = []

for k in range(1, len(S) + 1):

    Ak = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]

    errors.append(
        np.linalg.norm(A - Ak, "fro")
    )

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(S) + 1), errors)
plt.xlabel("k")
plt.ylabel("Frobenius Error")
plt.title("SVD Reconstruction Error")
plt.grid()
plt.show()
```